A Practical Guide to Spatial Data

UNIVERSITÀ
DI TORINO

Outline

1. **Coordinates, CRS & Reprojection**: The essential foundation.
2. **Types of Spatial Data**: A quick overview of data models.
3. **Vector Data**: Working with points, lines, and polygons.
4. **Raster Data**: Grids, pixels, and surfaces.
5. **Network Data**: Nodes, edges, and connectivity.
6. **Further Learning**: Curated tutorials and resources.

1. Coordinates, CRS & Reprojection

What is a Coordinate Reference System (CRS)?

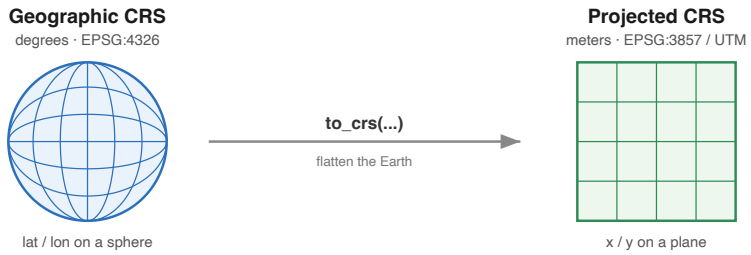
A CRS defines how coordinates map to a real-world location on Earth. Getting this wrong is the **#1 source of errors** in spatial analysis.

Geographic CRS

- Uses latitude and longitude on a spherical model.
- Units are in degrees.
- Good for global location, bad for measuring distance or area.
- **Example**: `EPSG:4326` (WGS 84)

Projected CRS

- Flattens the Earth onto a 2D surface.
- Units are in meters or feet.
- Essential for accurate distance, area, and buffer calculations.
- **Example**: `EPSG:3857` (Web Mercator), UTM Zones (e.g., `EPSG:32633`)



The Golden Rules of CRS: Rule 1

1. Always Know Your CRS: After loading data, immediately check the `.crs` attribute. If it's `None`, the data is unusable until you define it.

The Golden Rules of CRS: Rule 2

2. NEVER Guess: If the CRS is missing, find it in the metadata or from the data provider. Do not assume it's WGS 84.

The Golden Rules of CRS: Rule 3

3. Use `set_crs` vs. `to_crs` Correctly: - **`gdf.set_crs("EPSG:4326")`** : **Assigns** a CRS to data that has none. It **does not change the data**. Only use it if you are 100% certain what the CRS should be. - **`gdf.to_crs("EPSG:3857")`** : **Reprojects** the data, creating new coordinate values. Use this to transform data from one CRS to another.

Vector Reprojection with GeoPandas

This example shows how to load data, check its CRS, and reproject it for area calculations.

```
import geopandas as gpd

# 1. Load the data
gdf = gpd.read_file("data/berlin-neighbourhoods/berlin-neighbourhoods.geojson")

# 2. Always check the CRS
print(f"Original CRS: {gdf.crs}")

# 3. Reproject to a suitable projected CRS for measurements (e.g., UTM Zone 33N)
# This is CRITICAL for accurate area/distance calculations.
gdf_utm = gdf.to_crs("EPSG:32633")
print(f"New CRS: {gdf_utm.crs}")

# 4. Now, you can accurately calculate area in square meters
gdf_utm['area_sqkm'] = gdf_utm.geometry.area / 1_000_000
print(gdf_utm[['neighbourhood', 'area_sqkm']].head())
```

Library Guide: `pyproj`

`pyproj` wraps the **PROJ** library and is the engine every other tool calls when it has to move coordinates between systems — it is what makes a reprojection standards-compliant rather than a hand-rolled formula. Reach for it directly when you need to transform raw coordinate arrays or tuples without first building a GeoDataFrame, or when you want to inspect and validate a CRS definition.

The one habit worth internalizing: build a **`Transformer.from_crs(..., always_xy=True)`** once, reuse it for all your points (creating one per point in a loop is the classic performance trap), and remember that **`(lon, lat)`** order is **not** universal — **`always_xy=True`** pins it down.

Low-Level Reprojection with `pyproj` (setup)

Sometimes you only need to transform a few coordinates. `pyproj` is perfect for this. `geopandas` uses it under the hood.

```
from pyproj import CRS, Transformer

# 1) Define the source and destination CRS
# From WGS 84 (lon/lat) to Web Mercator (meters)
source_crs = CRS("EPSG:4326")
dest_crs = CRS("EPSG:3857")

# 2) Create a transformer
# always_xy=True ensures (lon, lat) -> (x, y) order
transformer = Transformer.from_crs(source_crs, dest_crs, always_xy=True)
```

Low-Level Reprojection with `pyproj` (transform)

```
# 3) Define a lon/lat point (San Diego)
lon, lat = -117.16, 32.71

# 4) Transform the point
x, y = transformer.transform(lon, lat)

print(f"Original (lon, lat): ({lon}, {lat})")
print(f"Projected (x, y): ({x:.2f}, {y:.2f})")
```

CRS & Reprojection: Further Learning

- **Pyproj Documentation**: The library that powers CRS transformations in Python.
  - https://pyproj4.github.io/pyproj/stable/
- **GeoPandas User Guide on CRS**: Practical examples and explanations.
  - https://geopandas.org/en/stable/docs/user_guide/projections.html
- **Pyproj Docs on CRS**: Understanding CRS objects.
  - https://pyproj4.github.io/pyproj/stable/api/crs/crs.html

2. Types of Spatial Data

A Quick Tour of Spatial Data Models

The first question on any project is which data model fits: **vector** for discrete objects with crisp edges, **raster** for a continuous field sampled on a grid.

Vector

Represents discrete objects with exact boundaries.

- **Points**: Locations (cities, sensors)
- **Lines**: Paths (rivers, roads)
- **Polygons**: Areas (countries, parks)

Use for: Administrative boundaries, infrastructure, points of interest.

Raster

Represents continuous phenomena on a grid of cells (pixels).

- Each pixel has a value.
- Examples: Elevation, temperature, satellite imagery.

Use for: Environmental data, imagery, surface analysis.

Networks (nodes + edges) are covered later for routing. *3D **point clouds** (LiDAR) are a fourth model, but out of scope in this course.*

3. Vector Data

Working with Points, Lines, and Polygons

Vector data is ideal for representing features with clear, defined shapes.

- **Core Python Libraries**:
  - `geopandas`: The essential tool. A `GeoDataFrame` is a `pandas.DataFrame` with a special `geometry` column.
  - `shapely`: Handles the geometry objects themselves.
  - `fiona`: For reading and writing data, powered by GDAL/OGR.

Library Guide: `geopandas`

geopandas is the workhorse of vector GIS in Python: a **GeoDataFrame** is just a **pandas.DataFrame** with a **geometry** column, so everything you know about filtering, grouping and joining tables still applies — plus **read_file** / **to_file**, **.to_crs**, **sjoin** and **.plot()**. It is the right default whenever your data fits comfortably in memory and you want EDA, spatial joins and quick cartography in a few lines.

Two habits keep it fast and correct: verify **gdf.crs** on load and reproject to a projected CRS *before* measuring area or distance, and let the spatial index (**.sindex**) or a bounding-box pre-filter do the heavy lifting before an expensive **sjoin** on large tables.

Library Guide: **shapely**

shapely is the geometry engine underneath GeoPandas — it owns the actual points, lines and polygons and the operations on them: constructive ops like **buffer**, **union**, **intersection**, and predicates like **contains** and **intersects**. You drop down to it directly when you need per-geometry logic that the high-level GeoPandas API does not express, or to repair geometries with **make_valid()**.
Do metric work (distances, areas, buffers) in a projected CRS rather than degrees, and when you test one static geometry against many candidates, wrap it with a **prepared** geometry to avoid re-parsing it on every check.

Library Guide: `fiona`

`fiona` is a Pythonic wrapper over GDAL/OGR for vector file I/O: it lets you open a specific layer, inspect its schema and CRS, and iterate features one at a time so you never have to load a huge dataset fully into memory. Reach for it when you need lower-level control than GeoPandas gives — custom driver options, streaming reads, or careful encoding handling for legacy data — always inside a `with fiona.open(...)` context.

Honest caveat: as of GeoPandas 1.0, `pyogrio` (a faster, vectorized GDAL binding) is the **default** I/O engine behind `read_file` / `to_file`. Fiona is now the legacy fallback — still useful for feature-by-feature streaming, but you rarely need to import it directly anymore.

Vector Formats & Trade-offs

- **Common Formats & Trade-offs**:
  - **GeoPackage (`.gpkg`)**: **Best choice**. An open standard, single-file database. Fast, flexible, and robust.
  - **Shapefile (`.shp`)**: Legacy format. Avoid if possible. Multi-file, column name limits, and other issues.
  - **GeoJSON (`.geojson`)**: Good for web applications, text-based, and easy to read. Not efficient for large datasets.

GeoPackage (`.gpkg`)

A GeoPackage is a single SQLite file that can hold many layers plus their spatial indexes, storing CRS and attributes without loss. Treat it as your default working format: write once, read any layer back by name.

```
import os, geopandas as gpd

os.makedirs("outputs", exist_ok=True)
# Write to a new GeoPackage layer
gdf = gpd.read_file("data/texas/texas.geojson")
gdf.to_file("outputs/data.gpkg", layer="texas", driver="GPKG")

# Read a specific layer back
gpkg = gpd.read_file("outputs/data.gpkg", layer="texas")
print(len(gpkg), gpkg.crs)
# → 254 EPSG:4326   (round-trip preserves all 254 counties and the CRS)
```

Shapefile (`.shp`)

The Shapefile is a 1990s format that travels as a bundle of sidecar files (**`.shp/.shx/.dbf/.prj`**) and silently truncates field names to 10 characters. Treat it as an interchange format you read from, not the system of record you write to.

```
import os, geopandas as gpd

os.makedirs("outputs", exist_ok=True)
gdf = gpd.read_file("data/texas/texas.geojson")
gdf[["NAME", "STATE_NAME", "HR90", "geometry"]].to_file("outputs/texas.shp")

shp = gpd.read_file("outputs/texas.shp")  # writes .shp/.shx/.dbf/.prj sidecars
print(list(shp.columns))
# → ['NAME', 'STATE_NAME', 'HR90', 'geometry']
# (longer attribute names would be cut to 10 chars, e.g. STATE_NAME stays, but a
#  hypothetical "population_density" would land as "populatio_")
```

Validate and fix geometries

Self-intersections and slivers make predicates and overlays misbehave, so screen for them on load. Prefer **make_valid()** — it repairs geometry topologically — over the old **buffer(0)** trick, which can quietly drop or distort parts.

```
import geopandas as gpd

gdf = gpd.read_file("data/berlin-neighbourhoods/berlin-neighbourhoods.geojson")
invalid = ~gdf.is_valid
gdf.loc[invalid, "geometry"] = gdf.loc[invalid, "geometry"].make_valid()
print(f"Fixed {int(invalid.sum())} invalid geometries")
# → Fixed 0 invalid geometries   (repaired count on messy data)
```

Spatial index & performance

- Use **gdf.sindex** (rtree/pygeos) to prefilter by bounding box.
- Avoid O(N×M) spatial joins by pre-clipping candidates.
- Read only needed columns; write GeoPackage for multi-layer outputs.

Aggregate and dissolve

dissolve is **groupby** for geometry: it merges all features sharing a key into one shape while aggregating their attributes. Here we melt 254 Texas counties up into the single state outline — and only measure area *after* reprojecting to an equal-area CRS, never in degrees.

```
import geopandas as gpd

gdf = gpd.read_file("data/texas/texas.geojson")  # 254 counties
out = gdf.to_crs("EPSG:6933")  # world cylindrical equal-area (m)

by_state = out.dissolve(by="STATE_NAME", aggfunc="sum")   # counties → state
by_state["area_km2"] = by_state.geometry.area / 1_000_000
print(by_state[["area_km2"]].round(0))
# →              area_km2
#   STATE_NAME
#   Texas       684886.0     (254 county polygons merged into 1)
```

Vector Recipe: Reading, Clipping, and Joining

The everyday vector pattern is *read polygons, build points, keep the points that fall inside, then label each point with the polygon it lands in*. Here we read Texas counties, turn an Airbnb listings CSV into a point layer, **clip** away stray geocodes outside the state, then **spatial-join** every listing to its county.

```
import geopandas as gpd
import pandas as pd

# 1. Read bundled polygons: Texas counties (EPSG:4326)
cols = ["NAME", "STATE_NAME", "geometry"]
counties = gpd.read_file("data/texas/texas.geojson")[cols]

# 2. Build a point layer from a CSV of Airbnb listings (lon/lat columns)
df = pd.read_csv("data/texas/listings.csv.gz")
pts = gpd.GeoDataFrame(
    df, geometry=gpd.points_from_xy(df.longitude, df.latitude), crs="EPSG:4326"
)

# 3. CLIP: keep only the points that fall inside the Texas counties
inside = gpd.clip(pts, counties)

# 4. JOIN: attach to each listing the county it sits in
listings_by_county = gpd.sjoin(
    inside, counties, how="inner", predicate="within")
print(len(pts), "->", len(inside))   # 5835 -> 5834 (one dropped)
print(listings_by_county["NAME"].value_counts().head(3))
```

Vector Data: Further Learning

- **GeoPandas Getting Started**: The official entry point.
  - https://geopandas.org/en/stable/getting_started.html
- **Plotting with GeoPandas**: Create beautiful maps.
  - https://geopandas.org/en/stable/docs/user_guide/mapping.html
- **Tutorial on Spatial Joins**: A fundamental concept explained well.
  - https://geopandas.org/en/stable/gallery/spatial_joins.html
- **Automating GIS Processes**: A great course with many vector data examples.
  - https://autogis-site.readthedocs.io/en/latest/

4. Raster Data

Working with Grids and Pixels

Raster data is a matrix of cells representing values over a continuous surface.

- **Key Properties**:
  - **Resolution**: The size of each pixel (e.g., 30 meters).
  - **Extent**: The bounding box of the entire grid.
  - **Bands**: A raster can have one or more layers (e.g., Red, Green, Blue bands in a color image).
  - **`nodata` value**: A special value assigned to pixels with no data.

- **Core Python Libraries**:
  - **`rasterio`**: The primary library for reading, writing, and manipulating raster files.
  - **`rioxarray`**: Integrates **`rasterio`** with **`xarray`** for powerful, labeled multi-dimensional array analysis (great for time-series!).

Raster formats in practice

- GeoTIFF: general-purpose workhorse; supports overviews and compression.
- COG (Cloud Optimized GeoTIFF): tiled + overviews for HTTP range requests.
- Bands & dtypes matter (uint8 imagery vs float32 surfaces); mind **nodata**.

Library Guide: `rasterio`

rasterio is the GDAL-backed library for reading and writing raster files, exposing the affine transform, CRS, **nodata** value and band metadata you need to treat a GeoTIFF as a georeferenced array. Reach for it when you want a single image or COG and need performant **windowed** reads — pulling only the pixels you care about instead of the whole grid.
Always open inside a **with rasterio.open(...) as src:** block so the file handle closes cleanly, respect **nodata** (cast to float and mask it before computing statistics), and choose resampling by data type — nearest for categorical labels, bilinear or cubic for continuous surfaces.

Library Guide: **rioxarray**

rioxarray marries Rasterio with **xarray**, giving you labeled, N-dimensional arrays with a CRS — the natural home for data-cube work across bands and time, with lazy Dask-backed computation for data too large to fit in memory. Its **.rio** accessor adds the geospatial verbs (**reproject**, **reproject_match**, **clip**, **to_raster**) on top of ordinary array math.
Reach for it instead of plain Rasterio when you have stacks or time series; the key discipline is to align grids with **reproject_match** before doing arithmetic between two rasters, and to be explicit about **nodata**.

Rasterio: open and windowed read

```
import os
import rasterio
from rasterio.windows import Window

ras_path = "data/sandiego/sandiego_tracts.tif"
if not os.path.exists(ras_path):
  print("Raster not found:", ras_path)
else:
  with rasterio.open(ras_path) as src:
    window = Window(0, 0, 512, 512)
    arr = src.read(1, window=window)
    print(arr.shape, src.res)
```

Resampling and reprojection notes

- Choose resampling by data type:
  - Nearest: categorical labels
  - Bilinear/Cubic: continuous surfaces
- Reproject with **`rioxarray.rio.reproject()`** (see earlier slide) or Rasterio's **`reproject`** API.

Raster Reprojection with `rioxarray`

`rioxarray` simplifies raster reprojection by handling the complex warping calculations for you.

```
import os
import rioxarray

os.makedirs("outputs", exist_ok=True)
ras_path = "data/sandiego/sandiego_tracts.tif"
if not os.path.exists(ras_path):
  print("Raster not found:", ras_path)
else:
  rds = rioxarray.open_rasterio(ras_path)
  rds_3857 = rds.rio.reproject("EPSG:3857")
  print("Reprojected CRS:", rds_3857.rio.crs)
  rds_3857.rio.to_raster("outputs/sandiego_tracts_3857.tif")
```

Raster Recipe: Reading, Masking, and Zonal Stats

This recipe calculates the mean raster value within each polygon.

```
import os
import geopandas as gpd
import rasterio
from rasterio.mask import mask
import numpy as np

ras_path = "data/sandiego/sandiego_tracts.tif"
vec_path = "data/berlin-neighbourhoods/berlin-neighbourhoods.geojson"
if not (os.path.exists(ras_path) and os.path.exists(vec_path)):
  print("Missing input(s):",
        ras_path, os.path.exists(ras_path), "|",
        vec_path, os.path.exists(vec_path))
else:
  # Load a raster and vector polygons
  with rasterio.open(ras_path) as src:
    polygons = gpd.read_file(vec_path)

    # IMPORTANT: Ensure polygons are in the same CRS as the raster
    polygons = polygons.to_crs(src.crs)
```

Raster Recipe (continued)

```
  # Iterate through each polygon to perform the mask
  results = []
  for geom in polygons.geometry:
    # Mask the raster with the polygon geometry
    out_image, _ = mask(src, [geom], crop=True)

    # Calculate the mean of the valid (non-nodata) pixels safely
    arr = out_image.astype('float32')
    nodata = src.nodata
    if nodata is not None:
      arr[arr == nodata] = np.nan
    mean_val = np.nanmean(arr)
    results.append(float(mean_val) if np.isfinite(mean_val) else None)

polygons['mean_raster_value'] = results
print(polygons[['neighbourhood', 'mean_raster_value']].head())
```

Raster Data: Further Learning

- **Rasterio Documentation & Cookbook**: Essential reading.
  - https://rasterio.readthedocs.io/en/latest/
- **rioxarray Usage Guide**: For when you need more advanced analysis.
  - https://corteva.github.io/rioxarray/stable/
- **Rasterio Quickstart Guide**: A great starting point for raster processing.
  - https://rasterio.readthedocs.io/en/latest/quickstart.html
- **Cloud-Optimized GeoTIFFs (COGs)**: The future of raster data access.
  - https://www.cogeo.org/

5. Network Data

Working with Nodes, Edges, and Connectivity

Spatial networks are graphs where nodes and/or edges have geographic locations. They are fundamental for routing, accessibility analysis, and logistics.

- **Use Cases**:
  - Calculating the shortest path between two points.
  - Finding all locations reachable within a 15-minute drive (isochrones).
  - Identifying the most critical intersections in a city.

- **Core Python Libraries**:
  - **OSMnx**: The best tool for downloading, analyzing, and visualizing street networks from OpenStreetMap.
  - **NetworkX**: A general-purpose library for graph analysis (centrality, pathfinding, etc.). **OSMnx** uses it under the hood.

Library Guide: `osmnx`

`osmnx` turns OpenStreetMap into a ready-to-analyze `networkx` graph in one call: `graph_from_place` downloads the streets, and it bundles geocoding, nearest-node queries, projection, plotting and `basic_stats`. Reach for it whenever you need a real street network for routing, accessibility or urban-form metrics without hand-parsing OSM.
Two practices matter: pick the `network_type` that matches your question (`drive` / `walk` / `bike` / `all`), and add edge speeds and travel times *before* routing on time. Cache graphs as GraphML so you are not re-downloading from the OSM servers on every run.

OSMnx essentials

- Download graphs from OSM, filter by mode (drive/walk/bike), plot in 1 line.
- Compute stats (**basic_stats**), simplify topology, save/load graphs.

```
import osmnx as ox
G = ox.graph_from_place("Kreuzberg, Berlin, Germany", network_type="walk")
ox.plot_graph(G, node_size=0, edge_color="gray")
```

Library Guide: `networkx`

networkx is the general-purpose graph library that OSMnx builds on: it provides the graph types and the algorithm zoo (shortest paths, centrality, connectivity, flows) independent of where the graph came from. Reach for it whenever your analysis is fundamentally about topology rather than geometry — and it is what you call directly once OSMnx has handed you a graph.
Street networks are directed and have parallel edges, so they live in a **MultiDiGraph**; store edge weights such as **length** or **travel_time** consistently and document their units, since the same **shortest_path** call gives very different routes depending on which weight you pass.

NetworkX essentials

```
import networkx as nx
G = nx.path_graph(5)
print(nx.shortest_path(G, 0, 4))  # [0,1,2,3,4]
```

Project your graph for metrics

```
import osmnx as ox
G = ox.graph_from_place("Kreuzberg, Berlin, Germany", network_type="drive")
Gp = ox.project_graph(G)  # meters-based CRS for accurate lengths
u, v, k, data = list(Gp.edges(keys=True, data=True))[0]
print(data.get("length"))
```

Routing with travel time

To route on *time* rather than distance, let OSMnx 2.x impute a speed limit per edge (**add_edge_speeds**) and derive **travel_time** from it (**add_edge_travel_times**). The subtle bug to avoid: **geocode** returns **(lat, lon)** in **degrees**, so the nodes you snap to must come from a graph in the **same** units — we keep **G** unprojected and pass **X=lon, Y=lat**.

```
import osmnx as ox

G = ox.graph_from_place("Kreuzberg, Berlin, Germany", network_type="drive")
G = ox.routing.add_edge_speeds(G)   # km/h from OSM maxspeed+highway
G = ox.routing.add_edge_travel_times(G)  # travel_time = length/speed

# geocode -> (lat, lon); snap on the SAME (unprojected) graph, X=lon, Y=lat
orig = ox.geocode("Görlitzer Bahnhof, Berlin")
dest = ox.geocode("Kottbusser Tor, Berlin")
orig_n = ox.nearest_nodes(G, X=orig[1], Y=orig[0])
dest_n = ox.nearest_nodes(G, X=dest[1], Y=dest[0])

route = ox.shortest_path(G, orig_n, dest_n, weight="travel_time")
print(len(route), "nodes")   # → e.g. 23 nodes along the fastest path
```

OSM tags and filters (1/2)

- Use **custom_filter** in **graph_from_place** / **graph_from_polygon** to restrict tags.
- Example: only primary/secondary roads; exclude private/service roads.

OSM tags and filters (2/2)

- Align network type with your analysis (walk vs bike vs drive).
- Validate assumptions by visual inspection and simple stats.

Network Recipe: Get a Street Network with OSMnx

This example downloads the street network for a neighborhood, calculates basic stats, and plots it.

```
import osmnx as ox

# 1. Define a place and download the street network
place_name = "Kreuzberg, Berlin, Germany"
# You can specify network_type as 'drive', 'walk', 'bike', 'all'
G = ox.graph_from_place(place_name, network_type='drive')

# 2. Plot the network
fig, ax = ox.plot_graph(G, node_size=0, edge_linewidth=0.5, edge_color='gray')

# 3. Calculate basic statistics
stats = ox.basic_stats(G)
print(f"Total street length: {stats['street_length_total'] / 1000:.2f} km")
```

Network Recipe (continued)

```
# 4. Find the shortest path (example)
origin = ox.geocode("Görlitzer Bahnhof, Berlin")
destination = ox.geocode("Kottbusser Tor, Berlin")
origin_node = ox.nearest_nodes(G, origin[1], origin[0])
dest_node = ox.nearest_nodes(G, destination[1], destination[0])

route = ox.shortest_path(G, origin_node, dest_node, weight='length')
fig, ax = ox.plot_graph_route(G, route, node_size=0, bgcolor="k")
```

Network Data: Further Learning

- **OSMnx Examples Gallery**: A fantastic collection of tutorials and advanced use cases.
  - https://osmnx.readthedocs.io/en/stable/user-reference/examples.html
- **NetworkX Tutorial**: Learn the fundamentals of graph theory and analysis.
  - https://networkx.org/documentation/stable/tutorial.html
- **Python GIS: Network Analysis**: A practical book combining theory and code.
  - https://pythongis.org/part-iv-network-analysis/

Setup: Python environment

What you'll set up

- A clean Python install (macOS)
- A project-local virtual environment (venv)
- Packages for this deck (GeoPandas, Rasterio, rioxarray, OSMnx, etc.)
- Jupyter for running notebooks
- VS Code as an editor (with Python + Jupyter extensions)

Base requirements (this repo)

These are the core packages in **notebooks/requirements.txt** used across the vector/raster parts of the deck:

```
geopandas
rasterio
rioxarray
xarray
pyproj
pandas
```

Extras (networks, notebooks, utilities) are listed in **notebooks/requirements-extras.txt** and include:

- osmnx, networkx
- jupyter, jupyterlab, ipykernel
- rtree, rasterstats, contextily, matplotlib
- laspy, open3d (optional point clouds)

Install Python on macOS

- Option A (easiest): Install from python.org
  - Download the latest macOS installer and follow prompts
  - After install, open Terminal and check:

```
python3 --version
pip3 --version
```

- Option B (Homebrew):

```
# Install Homebrew if you don't have it (https://brew.sh)
/bin/bash -c "$(curl -fsSL \
  https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Then install Python
brew install python
python3 --version
```

Create and activate a virtual environment

```
# Create a project folder (example) and enter it
mkdir -p ~/dev/spatial-work && cd ~/dev/spatial-work

# Create a virtual environment named .venv
python3 -m venv .venv

# Activate it (zsh)
source .venv/bin/activate

# Upgrade pip inside the venv
python -m pip install --upgrade pip
```

You should now see **(.venv)** in your terminal prompt. Use **deactivate** to exit.

Install required packages

- If you're in this repo, install the base packages from the provided requirements file:

```
# From the repo root
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
pip install -r notebooks/requirements.txt
```

- Then add the network + notebook tooling used in this deck:

```
pip install -r notebooks/requirements-extras.txt
```

- Alternatively, install everything explicitly (useful outside this repo):

```
pip install \
  geopandas shapely fiona rasterio rioxarray xarray matplotlib pyproj \
  rasterstats contextily osmnx networkx jupyter jupyterlab ipykernel rtree
```

If you hit build errors, first try: **`python -m pip install --upgrade pip setuptools wheel`**.

Verify the environment

```
import sys, importlib
pkgs = [
  ("geopandas", "gpd"),
  ("shapely", None),
  ("fiona", None),
  ("rasterio", None),
  ("rioxarray", None),
  ("osmnx", None),
  ("networkx", "nx"),
]
print("Python:", sys.version.split()[0])
for name, alias in pkgs:
    m = importlib.import_module(name)
    ver = getattr(m, "__version__", None)
    print(f"{name}: {ver}")
```

You should see version numbers printed without errors for each package.

Work with Jupyter Notebooks

```
# Start Jupyter Lab (recommended)
jupyter lab

# Or classic Notebook
jupyter notebook
```

- In the UI, ensure the kernel is your venv (Python in `.venv`).
- If the venv kernel isn't listed, install it and retry:

```
pip install ipykernel
python -m ipykernel install --user --name spatial-env \
  --display-name "Python (spatial-env)"
```

Then reopen the notebook and choose the new kernel.

Use VS Code as your editor

- Install VS Code and the extensions:
  - “Python” (Microsoft)
  - “Jupyter” (Microsoft)
- Open the project folder (the repo root)
- Select interpreter: Command Palette → “Python: Select Interpreter” → choose **.venv**
- For notebooks: pick the same kernel in the top-right kernel picker

Optional: Install the **code** command for launching VS Code from Terminal:

```
# VS Code → Command Palette → "Shell Command: Install 'code' command in PATH"
code .
```

Troubleshooting (macOS) 1/2

- Pip build errors for Rasterio/Fiona
  - Ensure Xcode Command Line Tools: **xcode-select --install**
  - Update build tools: **python -m pip install --upgrade pip setuptools wheel**
  - If needed, install libs: **brew install geos proj gdal** then retry **pip install**
  - If **rtree** fails, install spatial index libs: **brew install spatialindex** then **pip install rtree**
- Apple Silicon (M1/M2/M3)
  - Prefer Python from python.org or Homebrew (arm64). Use arm64 Terminal (not Rosetta) for consistency.
  - Wheels now exist for most geospatial packages; if a wheel isn't found, ensure Homebrew's **geos**, **proj**, and **gdal** are installed first.

Troubleshooting (macOS) 2/2

- Jupyter kernel not showing
  - Install the kernel: **`python -m ipykernel install --user --name spatial-env --display-name "Python (spatial-env)"`**
- Permission or PATH issues
  - Close/reopen Terminal after installs; confirm **`python3 --version`** and that **`(.venv)`** is active

Summary & Next Steps

- **CRS is everything**. Always check and correctly manage your coordinate systems.
- **Choose the right data model**: Vector for discrete features, Raster for continuous surfaces.
- **Master the core libraries**: `geopandas` and `rasterio` / `rioxarray` will handle 90% of your tasks.
- **Practice with real data**: Download data for your city or a region of interest and try to replicate these recipes.

Recommended Hands-on Path

1. Complete the **GeoPandas** "Getting Started" guide.
2. Work through the **rasterio** "Cookbook" examples.
3. Use **OSMnx** to analyze the street network of your own neighborhood.